

AI Agent Development Workshop

Cheat sheet -- keep this open on a second screen while you build. LangChain + Groq, Sept 9 2026

THE CORE LOOP

- 1 Write a Python function, decorate with **@tool**, write a clear docstring -- it's the ONLY thing the model reads to decide when/how to call it.
- 2 **llm.bind_tools([...])** -- tell the model which tools it may request.
- 3 **llm_with_tools.invoke(messages)** -- ask the model what to do.
- 4 Check **ai_message.tool_calls**: empty = it answered directly. Non-empty = it wants a tool run.
- 5 YOU run the actual function (the model never executes code) and get the result.
- 6 Feed the result back as a **ToolMessage**, keyed by **tool_call_id**.
- 7 Invoke again. Repeat 3-7 until **tool_calls** comes back empty -- that's the model deciding it's done.

PROMPT-ENGINEERING TEMPLATE

GOAL	What should the agent accomplish, in plain language? e.g. "Review every drone report and build a ranked dispatch list."
CONSTRAINTS	Exact rules/format it must follow: valid actions ("send rescue team" / "send drone" / "deprioritize"), valid severity levels, what NOT to flag, when to stop.
PLAN FIRST	Tell it to reason silently about each item BEFORE deciding -- "before you act, consider whether X or Y applies" -- not just react to the first thing it sees.

TOOLS & SETUP LINKS

Groq API key (free)	console.groq.com/keys
Groq model list	console.groq.com/docs/models -- check tool-calling support before you swap models
AI-assisted IDE	Google Antigravity (or your preferred AI pair-programmer)

STUCK? JUMP TO...

Environment won't validate / API key issues	-> phase_00_setup/
Can't get ANY tool call working at all	-> phase_01_first_agent/
Need to add a second capability / tool	-> phase_02_add_a_tool/
Agent forgets what was said two turns ago	-> phase_03_memory/
Need to process a whole list/queue, not just one item	-> phase_04_multi_step_task/
Stuck on YOUR track project specifically	-> tracks/<your track>/reference_solution/

Remember: every phase (00-04) runs standalone. Fell behind? Jump straight to whichever one matches your remaining time -- you don't need to finish the earlier ones first.